```java
public class GraphTheory {

    //Attributes
    private int vertexNumber;
    private boolean[][] adjacencyMatrix;

    //constructor
    public GraphTheory(int vertexNumber) {
        this.vertexNumber = vertexNumber;
        this.adjacencyMatrix = new boolean[vertexNumber][vertexNumber];
    }

    //methode for Adding edges
    public void AddEdge(int source, int destination) {
        adjacencyMatrix[source][destination] = true;
    }

    //checking if it is Reflexive
    public boolean Reflexive(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[i][i] != 1) {
                return false;
            }
        }
        return true;
    }

    //checking if it is Symmetric
    public boolean Symmetric(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix.length; j++) {
                if (matrix[i][j] != matrix[j][i]) {
                    return false;
                }
            }
        }
        return true;
    }

```

```
40      //checking if is Transitive
41      public boolean Transitive(int[][] matrix) {
42          int n = matrix.length;
43          int[][] result = new int[n][n];
44          for (int i = 0; i < n; i++) {
45              for (int j = 0; j < n; j++) {
46                  for (int k = 0; k < n; k++) {
47                      result[i][j] += matrix[i][k] *
    matrix[k][j];
48                  }
49              }
50          }
51          for (int i = 0; i < n; i++) {
52              for (int j = 0; j < n; j++) {
53                  if (result[i][j] > 0 && matrix[i][j
    ] == 0)
54                      return false;
55              }
56          }
57          return true;
58      }
59      //checking if it is Acyclic
60      public boolean Acyclicity(int[][] matrix) {
61          boolean [] visited = new boolean [matrix.
    length];
62          boolean[] recursion = new boolean[matrix.
    length];
63          for (int i = 0; i <matrix.length; i++) {
64              if (hasCycle(matrix,1, visited, recursion
    )) {
65                  return false;
66              }
67          }
68          return true;
69      }
70
71      //checking if it has Cycle
72      private boolean hasCycle(int[][] matrix, int
    vertex, boolean[] visited,  boolean[] recursionStack
    ) {
73          if (recursionStack[vertex])
```

```
 74              return true;
 75          if (visited[vertex])
 76              return false;
 77          visited[vertex] = true;
 78          recursionStack[vertex] = true;
 79          for (int i = 0; i < matrix.length; i++) {
 80              if (matrix[vertex][i] == 1 && hasCycle(
    matrix, i, visited, recursionStack))
 81                  return true;
 82          }
 83          recursionStack[vertex] = false;
 84          return false;
 85      }
 86      //checking if it is connected
 87      public boolean Connectivity(int[][] matrix) {
 88          boolean[] visited= new boolean[matrix.length
    ];
 89          DFS(matrix, 0, visited);
 90
 91          for (int i = 0; i < matrix.length; i++) {
 92              if (!visited[i]) {
 93                  return false;
 94              }
 95          }
 96          return true;
 97      }
 98
 99      private static void DFS(int[][] matrix, int
    vertex, boolean[] visited) {
100          visited[vertex] = true;
101          for (int i = 0; i < matrix.length; i++) {
102              if (matrix[vertex][i] ==1 && !visited[i
    ]) {
103                  DFS(matrix, i, visited);
104              }
105          }
106      }
107      //checking if it has Eulerian path
108      public boolean EulerianPath(int[][] matrix) {
109          int n = matrix.length;
110          int[] degree = new int[n];
```

```
111         int odd = 0;
112         for (int i = 0; i < n; i++) {
113             for (int j = 0; j < n; j++) {
114                 degree[i] += matrix[i][j];
115             }
116             if (degree[i] % 2 != 0)
117                 odd++;
118         }         if (odd == 0 || odd == 2)
119             return true;
120         else
121             return false;
122             }
123
124         }
```